



Yggdrasil ERP — Database Architecture

Mimir Labs Technical Publication

Document:	ML-WP-005	Version:	1.1
Author:	Christopher Gaither	Date:	April 2026
Classification:	Public		

0.1 Executive Summary

The Yggdrasil ERP database is designed as a governed persistence layer rather than a passive storage engine. It provides multi-tenant isolation, auditable data mutation, deterministic lifecycle states, and predictable query performance across the operational domains of a modern enterprise system.

The implementation uses PostgreSQL as the underlying datastore, but the architectural focus is not the database technology itself. The database functions as the canonical operational record supporting Yggdrasil's state-machine execution model and the broader Mimir Labs architecture.

This paper describes how the schema, indexing strategy, multi-tenant controls, audit infrastructure, and schema-management model work together to maintain coherent enterprise data at scale.

0.2 1. Architectural Role of the Database

In Yggdrasil, the database is not merely a persistence target for application objects. It is a controlled boundary that preserves operational truth.

Three responsibilities define the role of the database layer:

First, it must provide a reliable representation of enterprise state across multiple operational domains including manufacturing, finance, quality, logistics, and service.

Second, it must enforce structural integrity and tenant isolation so that application-level defects cannot compromise organizational boundaries.

Third, it must maintain a complete history of operational changes so that lifecycle transitions remain observable and auditable.



These properties allow the system to support both operational execution and post-hoc analysis without introducing reconciliation ambiguity.

0.3 2. Technology Foundation

The database layer is implemented on PostgreSQL. The choice was driven by features that directly support ERP-class systems.

Row-level security provides database-enforced tenant isolation. JSONB storage enables flexible representation of structured artifacts such as audit payloads and form definitions. Partial indexes allow the system to optimize queries around active operational data rather than historical records. ENUM types allow the database to enforce lifecycle value constraints directly within the schema.

Together, these capabilities allow the persistence layer to actively enforce system invariants rather than relying solely on application code.

0.4 3. Schema Organization

The Yggdrasil schema spans the operational domains typically required by an enterprise system. These include customer relationship management, sales, purchasing, manufacturing, warehouse management, finance, quality assurance, service operations, logistics, projects, and human resources.

Supporting infrastructure tables manage tenancy, user identity, permissions, attachments, workflow instances, integration endpoints, and system configuration. Additional structures support form definitions, asset tracking, MRP planning artifacts, and multi-currency accounting. The schema also includes governance structures: the state-engine runtime tables (`state_entity_types`, `state_transitions`, `state_transition_constraints`) read by the State Constraint Engine, and the ROPE governance tables (`rope_decisions`, `rope_decision_artifacts`, `rope_evaluation_log`, `rope_named_conditions`, and related) that support Runtime Operational Policy Enforcement.

The schema comprises 345 tables, 866 indexes, and 6 views. Despite this scale, its structure follows a consistent naming and relationship pattern. Entities are organized by module, primary keys are universally represented as UUIDs, foreign key relationships follow explicit naming conventions, and lifecycle timestamps appear consistently across operational tables.

This uniformity is intentional. The schema is designed to remain legible and predictable even as additional modules are introduced.

0.5 4. Multi-Tenancy Model

Yggdrasil uses a shared database and shared schema model with logical tenant isolation. Most operational tables include a tenant identifier that scopes each record to its owning organization.

While this is a common multi-tenant pattern, Yggdrasil strengthens it with database-level enforce-



ment. Row-level security policies ensure that every query automatically filters data by tenant context, and FORCE ROW LEVEL SECURITY is applied so that the policies bind even for table owners. The application sets the active tenant identifier for each transaction, and PostgreSQL applies isolation rules to all read and write operations.

This defense-in-depth approach ensures that tenant boundaries remain intact even if application logic fails to include appropriate filters.

The tenants table acts as the root of the organizational hierarchy, storing identity information and configuration data for each environment.

0.6 5. Lifecycle and Status Modeling

Many ERP failures originate from loosely governed status fields. In Yggdrasil, lifecycle states are treated as first-class constraints within the schema.

Custom ENUM types define valid lifecycle values for major entities such as orders, work orders, engineering changes, financial journals, quality reports, and service tickets. These ENUM definitions work in conjunction with the server's State Constraint Engine, which reads the state-engine runtime tables and the ROPE governance tables to enforce Runtime Operational Policy Enforcement inside the write transaction, ensuring that transitions occur only within defined lifecycle graphs.

By enforcing lifecycle constraints at both the application and database layers, the system prevents invalid states from entering operational history.

0.7 6. Audit and Change Tracking

Operational traceability is a core property of the Yggdrasil architecture. The database records every meaningful mutation through a centralized audit log.

Each audit record captures the acting user, tenant context, action type, affected table, entity identifier, and before-and-after data snapshots stored in JSON format. Status transitions are recorded explicitly so that lifecycle histories remain visible even when other fields change.

Critical audit and financial structures are made tamper-evident at the database level. The audit_change_log and ledger_entries tables carry database triggers that reject UPDATE, DELETE, and TRUNCATE operations, rendering their history effectively append-only. This design provides a durable historical record of operational activity while remaining flexible enough to represent diverse entity structures.

Soft deletion is also supported across entity tables. Instead of removing records entirely, deletion operations mark records as inactive while preserving them for historical and audit purposes.



0.8 7. Index and Performance Strategy

The indexing strategy focuses on supporting the most common ERP access patterns: tenant-scoped queries, lifecycle filtering, chronological reporting, and relationship traversal.

Composite indexes typically begin with tenant identifiers so that queries remain efficient within large multi-tenant datasets. Partial indexes limit storage and lookup overhead by indexing only operationally active records. Date-ordered indexes support timeline-based queries such as recent transactions, postings, or workflow activity.

These patterns allow the system to scale without requiring overly aggressive hardware resources while maintaining predictable query latency.

0.9 8. Relationship Architecture

Operational modules are connected through explicit foreign key relationships. These relationships represent real enterprise workflows such as the progression from quotes to orders, orders to invoices, work orders to operations, or purchase orders to receipts.

Cascade rules are applied selectively. Child line items typically cascade from parent documents, while cross-module references use restrictive deletion rules to prevent accidental removal of critical operational history.

The server exposes dedicated relationship endpoints that traverse these connections, enabling client applications to navigate operational flows without constructing complex joins themselves.

0.10 9. Schema Management Model

In the current pre-production phase, the schema is managed as a single canonical artifact rather than through a runtime migration system. All structural changes (new tables, column additions, ENUM extensions, and index changes) are made directly in one authoritative schema file. Demo and test databases are dropped and recreated from that file after each change, so the schema definition and the running databases never diverge.

There is no sequential migration system executed during server startup. The migrations directory and its associated lint tooling remain in the repository but are inactive. They become active again only once a paying customer holds production data that must be preserved across upgrades, at which point evolution shifts to versioned, additive migrations rather than full rebuilds.

This model is deliberately chosen for the pre-production period: keeping a single source of truth eliminates drift between migration history and actual structure, and rebuilding from the canonical file guarantees that every environment reflects the latest schema exactly.



0.11 10. Flexible Data Structures

Although most operational entities use strongly typed relational structures, certain artifacts require flexible representation. These include form definitions, integration payloads, configuration settings, and audit snapshots.

JSONB columns support these cases without introducing uncontrolled schema sprawl. PostgreSQL's JSON operators allow the system to query inside these structures when necessary while retaining relational performance characteristics.

The result is a hybrid model that combines strict relational governance with controlled flexibility.

0.12 11. Reporting and Derived Views

Several database views provide aggregated operational perspectives across modules. These views support reporting scenarios such as inventory valuation, order fulfillment status, financial summaries, work order progress, and quality metrics.

By encapsulating complex joins and calculations inside views, the database provides stable reporting interfaces while shielding client applications from underlying schema complexity.

This approach also allows reporting logic to evolve independently from the transactional schema.

0.13 12. Backup and Recovery

Operational resilience requires predictable backup and restoration capabilities. The database layer supports scheduled full backups with encryption and integrity verification. Retention policies allow deployments to adjust storage requirements according to regulatory and operational needs.

Recovery procedures support both full restoration and point-in-time recovery through PostgreSQL's write-ahead logging mechanisms.

Administrative endpoints in the server provide controlled access to backup management and restoration workflows.

0.14 13. Schema Evolution Philosophy

The long-term stability of the Yggdrasil platform depends on disciplined schema evolution. Several guiding principles govern this process.

Additive changes are preferred so that new capabilities do not disrupt existing workflows. ENUM expansions occur by appending values rather than modifying existing definitions. Once production data exists and versioned migrations resume, structural removals are staged gradually across multiple releases.

Most importantly, the schema itself represents the operational expression of canonical enterprise



meaning. External tools and migration utilities must adapt to this structure rather than redefining it.

Within the broader Mimir Labs architecture, the schema functions as the operational counterpart to the canonical semantic model maintained by Mimirbrunnr.

0.15 Conclusion

The Yggdrasil database architecture is designed to preserve operational coherence rather than simply store records. Tenant isolation, lifecycle enforcement, audit capture, and disciplined schema evolution ensure that enterprise data remains trustworthy over time.

By combining strong relational governance with selective flexibility, the database layer supports the deterministic execution model implemented by the Yggdrasil server while remaining compatible with integration, reporting, and future semantic tooling across the Mimir Labs stack.

Copyright 2026 Mimir Labs. All rights reserved.

Mimir Labs LLC | Harrisburg, PA

© 2024–2026 Mimir Labs LLC. All rights reserved.

This document contains proprietary information belonging to Mimir Labs LLC. No part of this publication may be reproduced, distributed, or transmitted in any form without the prior written permission of Mimir Labs LLC. The information herein is provided “as is” without warranty of any kind. Product names and trademarks referenced in this document are the property of their respective owners.